

Problem Statement

Dalam proses pengembangan aplikasi e-commerce maupun sistem inventori, data produk merupakan komponen utama yang harus dikelola dengan baik. Pengelolaan data produk secara manual atau tanpa API menyebabkan proses pencarian, penambahan, pembaruan, dan penghapusan data menjadi kurang efisien serta sulit diintegrasikan dengan aplikasi frontend.

Oleh karena itu, diperlukan sebuah REST API yang dapat menyediakan layanan pengelolaan data produk secara terstruktur. API ini memungkinkan aplikasi client mengakses data produk melalui endpoint yang sesuai dengan standar HTTP sehingga proses pertukaran data menjadi lebih mudah, cepat, dan konsisten.

Tujuan Mini Project

Mini project ini bertujuan untuk:

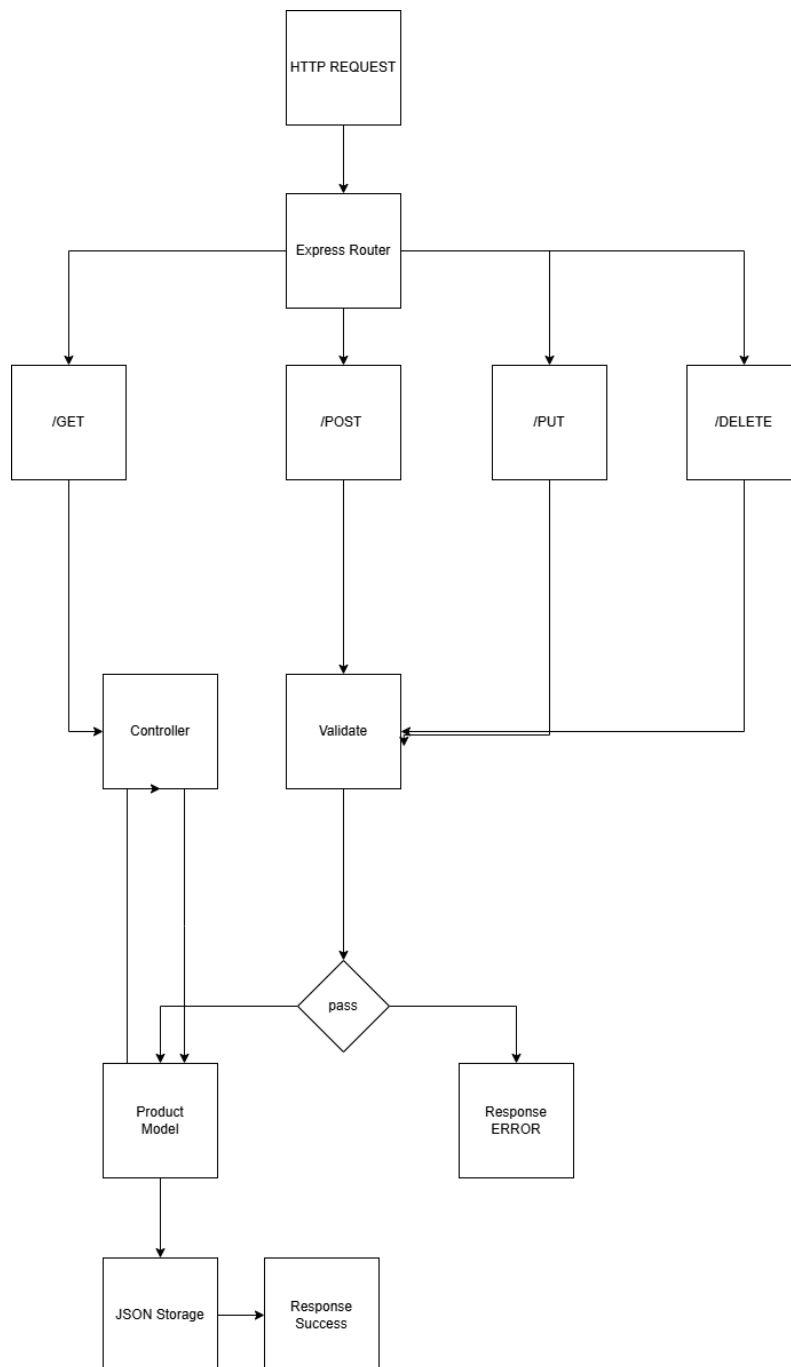
1. Mengembangkan REST API menggunakan Express.js untuk mengelola data produk.
2. Mengimplementasikan operasi CRUD (Create, Read, Update, Delete) pada data produk.
3. Menerapkan endpoint API yang menggunakan metode HTTP seperti GET, POST, PATCH, dan DELETE.
4. Melakukan pengujian endpoint menggunakan Postman untuk memastikan seluruh fungsi API berjalan dengan baik.
5. Menerapkan struktur kode yang modular agar mudah dipelihara dan dikembangkan di masa mendatang.

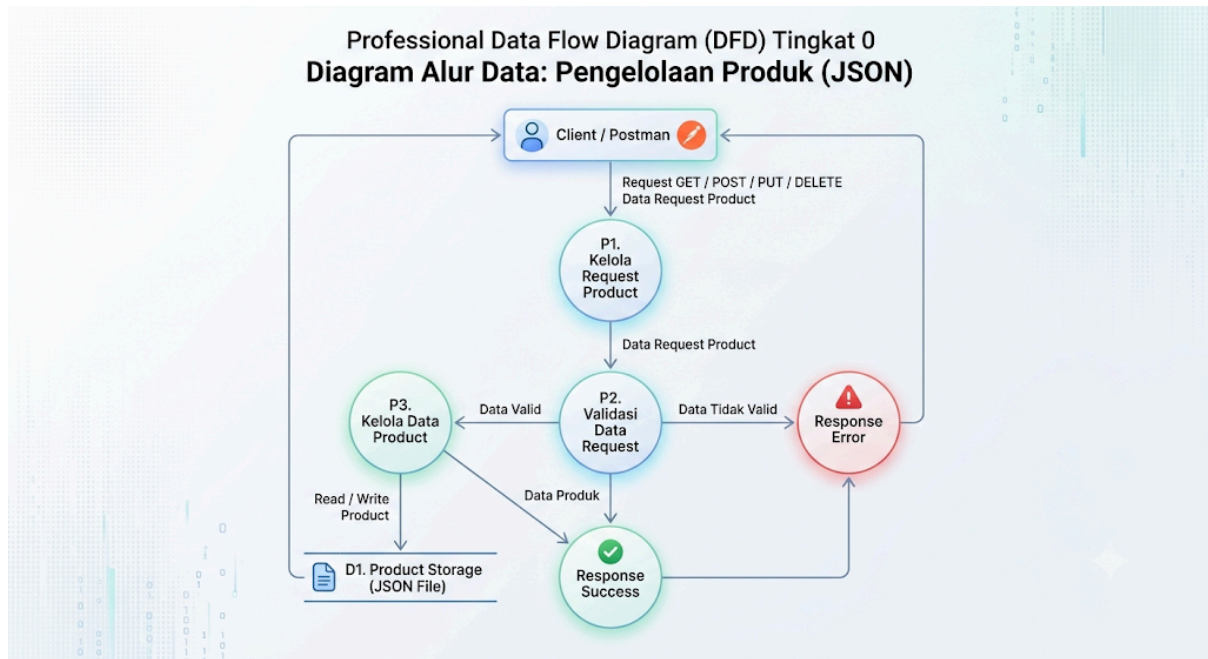
Alasan Memilih Use Case

Use case pengelolaan produk dipilih karena merupakan salah satu implementasi REST API yang paling umum digunakan pada berbagai aplikasi, seperti e-commerce, sistem kasir, inventori, dan marketplace. Melalui use case ini, seluruh konsep dasar REST API dapat diterapkan, mulai dari pembuatan endpoint, penggunaan metode HTTP, validasi data, hingga pengelolaan response.

Selain itu, data produk memiliki struktur yang jelas, seperti nama produk, harga, kategori, stok, dan deskripsi, sehingga cocok digunakan sebagai studi kasus untuk memahami proses komunikasi antara client dan server menggunakan [Express.js](#).

Architecture Dan Diagram Data Flow





Protocol Selection (REST API)

REST (Representational State Transfer) dipilih sebagai protokol komunikasi pada mini project ini karena sesuai dengan kebutuhan aplikasi yang berfokus pada pengelolaan data produk melalui operasi CRUD (Create, Read, Update, Delete). REST API memanfaatkan protokol HTTP sehingga memungkinkan komunikasi antara client dan server dilakukan dengan cara yang sederhana, efisien, dan mudah diimplementasikan.

Pada mini project ini, REST API digunakan untuk menyediakan endpoint yang dapat diakses oleh client menggunakan metode HTTP, yaitu **GET** untuk mengambil data produk, **POST** untuk menambahkan produk baru, **PUT** untuk memperbarui data produk, dan **DELETE** untuk menghapus data produk. Seluruh pertukaran data dilakukan dalam format JSON sehingga memudahkan proses integrasi dengan aplikasi frontend maupun tools pengujian seperti Postman.

REST API dipilih karena merupakan standar yang umum digunakan dalam pengembangan aplikasi web, memiliki dokumentasi yang luas, mudah dikembangkan, serta mendukung arsitektur yang bersifat stateless sehingga setiap request diproses secara independen. Berdasarkan kebutuhan mini project yang hanya memerlukan layanan CRUD pada data produk, REST API menjadi pilihan yang paling sesuai dibandingkan protokol lain seperti WebSocket, MQTT, maupun gRPC yang lebih ditujukan untuk kebutuhan komunikasi real-time, IoT, atau microservices.

Endpoint/Action Design

REST API pada mini project ini dirancang untuk menyediakan layanan pengelolaan data produk melalui operasi **Create, Read, Update, dan Delete (CRUD)**. Setiap endpoint menggunakan metode HTTP yang sesuai dengan fungsinya sehingga komunikasi antara client dan server dapat berlangsung secara terstruktur dan mengikuti standar REST API. Seluruh data yang dipertukarkan menggunakan format **JSON (JavaScript Object Notation)** agar mudah diproses oleh berbagai platform maupun aplikasi frontend.

Setiap endpoint memiliki spesifikasi yang meliputi metode HTTP (*method*), alamat endpoint (*path*), *header* yang digunakan, *request body* untuk operasi yang memerlukan input data, *response body* yang dikembalikan oleh server, serta *HTTP status code* sebagai indikator keberhasilan atau kegagalan proses. Dengan adanya perancangan endpoint yang jelas, proses pengembangan, pengujian menggunakan Postman, serta integrasi dengan aplikasi client menjadi lebih mudah dan konsisten.

Rancangan endpoint yang diimplementasikan pada mini project ini dapat dilihat pada tabel berikut.

No	Method	Path	Header	Request Body	Response Body	Expected Status Code
1	GET	/api/products	Content-Type: application/json	-	Daftar seluruh data produk	200 OK
2	GET	/api/products/:id	Content-Type: application/json	-	Detail produk berdasarkan ID	200 OK, 404 Not Found
3	POST	/api/products	Content-Type: application/json	Data produk baru	Data produk yang berhasil ditambahkan	201 Created, 400 Bad Request
4	PUT	/api/products/:id	Content-Type: application/json	Data produk yang diperbarui	Data produk setelah diperbarui	200 OK, 400 Bad Request, 404 Not Found
5	DELETE	/api/products/:id	Content-Type: application/json	-	Pesan berhasil menghapus data	200 OK, 404 Not Found

Evidence Success

success test

Comp2 UAS > Get Product By Id

GEThttp://localhost:3000/api/products/0fe6f2cf-5388-4d66-8f3f-2180806fb2be

Send

DocsParamsAuthorizationHeaders6BodyScriptsSettingsCookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

BodyCookiesHeaders19Test Results200 OK8 ms1.11 KBSave Response

JSONPreviewVisualize

```
1 {
2   "success": true,
3   "data": {
4     "id": "0fe6f2cf-5388-4d66-8f3f-2180806fb2be",
5     "name": "Keyboard Mechanical",
6     "price": 750000,
7     "stock": 25,
8     "category": "electronics",
9     "createdAt": "2026-07-07T15:10:34.446Z",
10    "updatedAt": "2026-07-07T15:10:34.447Z"
11  }
12 }
```

PATCHhttp://localhost:3000/api/products/0fe6f2cf-5388-4d66-8f3f-2180806fb2be

Send

DocsParamsAuthorizationHeaders8BodyScriptsSettingsCookies

noneform-datax-www-form-urlencodedit rawbinaryGraphQLJSONSchemaBeautify

1 {
2 "name": "Mesin cuci",
3 "price": 7500,
4 "stock": 25,
5 "category": "electronics"
6 }

BodyCookiesHeaders19Test Results200 OK9 ms1.1 KBSave Response

JSONPreviewVisualize

```
1 {
2   "success": true,
3   "data": {
4     "id": "0fe6f2cf-5388-4d66-8f3f-2180806fb2be",
5     "name": "Mesin cuci",
6     "price": 7500,
7     "stock": 25,
8     "category": "electronics",
9     "createdAt": "2026-07-07T15:10:34.446Z",
10    "updatedAt": "2026-07-07T15:14:25.567Z"
11  }
12 }
```

failure test

GET

http://localhost:3000/api/products/9820504d-e502-44bf-8e6c-2a85edf86ba9

Send

Docs

Params

Authorization

Headers

6

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

Cookies

Headers

19

Test Results

404 Not Found

15 ms

1015 B

Save Response

...

{ } JSON

Preview

Debug with AI

```
1 {
2   "success": false,
3   "message": "Product dengan id 9820504d-e502-44bf-8e6c-2a85edf86ba9 tidak ditemukan"
4 }
```

Observability/traffic evidence

Terapkan suatu filter tampilan ... Ctrl

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	:::1	:::1	TCP	76	59893 → 3000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
2	0.000181900	:::1	:::1	TCP	76	3000 → 59893 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
3	0.000182400	:::1	:::1	TCP	64	59893 → 3000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
4	0.001398200	:::1	:::1	TCP	76	59894 → 3000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
5	0.001523500	:::1	:::1	TCP	76	3000 → 59894 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
6	0.001566100	:::1	:::1	TCP	64	59894 → 3000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
7	0.001720600	:::1	:::1	TCP	64	59893 → 3000 [FIN, ACK] Seq=1 Ack=1 Win=65280 Len=0
8	0.001739700	:::1	:::1	TCP	64	3000 → 59893 [FIN, ACK] Seq=1 Ack=2 Win=65280 Len=0
9	0.002174900	:::1	:::1	HTTP	277	GET /api/products HTTP/1.1
10	0.002201400	:::1	:::1	TCP	64	3000 → 59894 [ACK] Seq=1 Ack=214 Win=65280 Len=0
11	0.005817900	:::1	:::1	TCP	64	3000 → 59893 [FIN, ACK] Seq=1 Ack=2 Win=65280 Len=0
12	0.005884900	:::1	:::1	TCP	64	59893 → 3000 [ACK] Seq=2 Ack=2 Win=65280 Len=0
13	0.012549300	:::1	:::1	HTTP/1	1400	HTTP/1.1 200 OK , JSON (application/json)
14	0.012620500	:::1	:::1	TCP	64	59894 → 3000 [ACK] Seq=214 Ack=1337 Win=64000 Len=0
15	1.015445200	:::1	:::1	TCP	65	[TCP Keep-Alive] 59894 → 3000 [ACK] Seq=213 Ack=1337 Win=64000 Len=1
16	1.015509300	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 59894 [ACK] Seq=1337 Ack=214 Win=65280 Len=0 SLE=213 SRE=214
17	2.021640400	:::1	:::1	TCP	65	[TCP Keep-Alive] 59894 → 3000 [ACK] Seq=213 Ack=1337 Win=64000 Len=1
18	2.021682900	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 59894 [ACK] Seq=1337 Ack=214 Win=65280 Len=0 SLE=213 SRE=214
19	3.035905700	:::1	:::1	TCP	65	[TCP Keep-Alive] 59894 → 3000 [ACK] Seq=213 Ack=1337 Win=64000 Len=1
20	3.035943900	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 59894 [ACK] Seq=1337 Ack=214 Win=65280 Len=0 SLE=213 SRE=214
21	4.036277500	:::1	:::1	TCP	65	[TCP Keep-Alive] 59894 → 3000 [ACK] Seq=213 Ack=1337 Win=64000 Len=1
22	4.036325900	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 59894 [ACK] Seq=1337 Ack=214 Win=65280 Len=0 SLE=213 SRE=214

Flags: 0x018 (PSH, ACK)
Window: 255
[calculated window size: 65280]
[window size scaling factor: 256]
Checksum: 0x5263 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
[Timestamps]
[SEQ/ACK analysis]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]
TCP payload (213 bytes)
Hypertext Transfer Protocol
GET /api/products HTTP/1.1\r\nUser-Agent: PostmanRuntime/7.54.0\r\nAccept: */*\r\nPostman-Token: 2b4b9645-a897-4876-8b09-8501bda62b18\r\nHost: localhost:3000\r\nAccept-Encoding: gzip, deflate, br\r\nConnection: keep-alive\r\n\r\n[Response in frame: 13]
[Full request URI: http://localhost:3000/api/products]

0000 18 00 00 00 60 02 fa a7 00 e9 06 80 00 00 00 00
0010 00 00 00 00 00 00 00 00 00 00 01 00 00 00 00
0020 00 00 00 00 00 00 00 00 00 00 01 e9 f6 0b b8
0030 74 85 fb 53 4d 27 df f5 50 18 00 ff 52 63 00 00
0040 47 45 54 20 2f 61 70 69 2f 70 72 6f 64 75 63 74
0050 73 20 48 54 50 2f 31 2e 21 0d 0a 55 73 65 72
0060 2d 41 67 65 6e 7a 3a 20 50 6f 73 74 6d 61 6e 52
0070 75 6e 74 69 6d 65 2f 37 2e 35 34 2e 30 0d 0a 41
0080 63 63 65 70 74 3a 20 2a 2f 2a 0d 0a 50 6f 73 74
0090 6d 61 6e 2d 54 6f 6b 65 6e 3a 20 32 62 34 62 39
00a0 36 34 35 2d 61 38 39 37 2d 34 38 37 36 2d 38 62
00b0 30 39 2d 38 35 30 31 62 64 61 36 32 62 31 38 0d
00c0 0a 48 6f 73 74 3a 20 6c 6f 63 61 6c 68 6f 73 74
00d0 3a 33 30 30 30 0d 0a 41 63 63 65 70 74 2d 45 6e
00e0 63 6f 64 69 6e 67 3a 20 67 7a 69 70 2c 20 64 65
00f0 66 6c 61 74 65 2c 20 62 72 0d 0a 43 6f 6e 6e 65
0100 63 74 69 6f 6e 3a 20 6b 65 65 70 2d 61 6c 69 76
0110 65 0d 0a 0d 0a

Adapter for loopback traffic capture: <live capture in progress>

Paket: 28

Profile: Default

Terapkan suatu filter tampilan... <Ctrl>

No.	Time	Source	Destination	Protocol	Length	Info
94	312.931551500	:::1	:::1	TCP	64	60520 → 3000 [FIN, ACK] Seq=1 Ack=1 Win=65280 Len=0
95	312.931574500	:::1	:::1	TCP	64	3000 → 60520 [ACK] Seq=1 Ack=2 Win=65280 Len=0
96	312.931914000	:::1	:::1	HTTP	314	GET /api/products/0fe6f2cf-5388-4d66-8f3f-2180806fb2be HTTP/1.1
97	312.931938900	:::1	:::1	TCP	64	3000 → 60521 [ACK] Seq=1 Ack=251 Win=65280 Len=0
98	312.932438500	:::1	:::1	TCP	64	3000 → 60520 [FIN, ACK] Seq=1 Ack=2 Win=65280 Len=0
99	312.932491600	:::1	:::1	TCP	64	60520 → 3000 [ACK] Seq=2 Ack=2 Win=65280 Len=0
100	312.946685700	:::1	:::1	HTTP/1.1	1188	HTTP/1.1 200 OK, JSON (application/json)
101	312.946744100	:::1	:::1	TCP	64	60521 → 3000 [ACK] Seq=251 Ack=1125 Win=64256 Len=0
102	312.960843600	:::1	:::1	TCP	65	[TCP Keep-Alive] 60521 → 3000 [ACK] Seq=250 Ack=1125 Win=64256 Len=1
103	313.960886500	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 60521 [ACK] Seq=1125 Ack=251 Win=65280 Len=0 SLE=250 SRE=251
104	314.973477000	:::1	:::1	TCP	65	[TCP Keep-Alive] 60521 → 3000 [ACK] Seq=250 Ack=1125 Win=64256 Len=1
105	314.973515100	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 60521 [ACK] Seq=1125 Ack=251 Win=65280 Len=0 SLE=250 SRE=251
106	315.983965900	:::1	:::1	TCP	65	[TCP Keep-Alive] 60521 → 3000 [ACK] Seq=250 Ack=1125 Win=64256 Len=1
107	315.984005600	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 60521 [ACK] Seq=1125 Ack=251 Win=65280 Len=0 SLE=250 SRE=251
108	316.998410200	:::1	:::1	TCP	65	[TCP Keep-Alive] 60521 → 3000 [ACK] Seq=250 Ack=1125 Win=64256 Len=1
109	316.998449200	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 60521 [ACK] Seq=1125 Ack=251 Win=65280 Len=0 SLE=250 SRE=251
110	318.005796800	:::1	:::1	TCP	65	[TCP Keep-Alive] 60521 → 3000 [ACK] Seq=250 Ack=1125 Win=64256 Len=1
111	318.005930900	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 60521 [ACK] Seq=1125 Ack=251 Win=65280 Len=0 SLE=250 SRE=251
112	318.957677000	:::1	:::1	TCP	64	3000 → 60521 [FIN, ACK] Seq=1125 Ack=251 Win=65280 Len=0
113	318.957759400	:::1	:::1	TCP	64	60521 → 3000 [ACK] Seq=251 Ack=1126 Win=64256 Len=0
114	318.958259000	:::1	:::1	TCP	64	60521 → 3000 [FIN, ACK] Seq=251 Ack=1126 Win=64256 Len=0
115	318.958330700	:::1	:::1	TCP	64	3000 → 60521 [ACK] Seq=1126 Ack=252 Win=65280 Len=0

Flags: 0x018 (PSH, ACK)
Window: 255
[Calculated window size: 65280]
[Window size scaling factor: 256]
Checksum: 0x01f0 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
[Timestamps]
[SQ/ACK analysis]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]
TCP payload (250 bytes)
Hypertext Transfer Protocol
GET /api/products/0fe6f2cf-5388-4d66-8f3f-2180806fb2be HTTP/1.1\r\nUser-Agent: PostmanRuntime/7.54.0\r\nAccept: */*\r\nPostman-Token: 6af982c8-7715-460e-8211-ad3e76c33fa\r\nHost: localhost:3000\r\nAccept-Encoding: gzip, deflate, br\r\nConnection: keep-alive\r\n\r\n[Response in frame: 100]
[Full request URI: http://localhost:3000/api/products/0fe6f2cf-5388-4d66-8f3f-2180806fb2be]

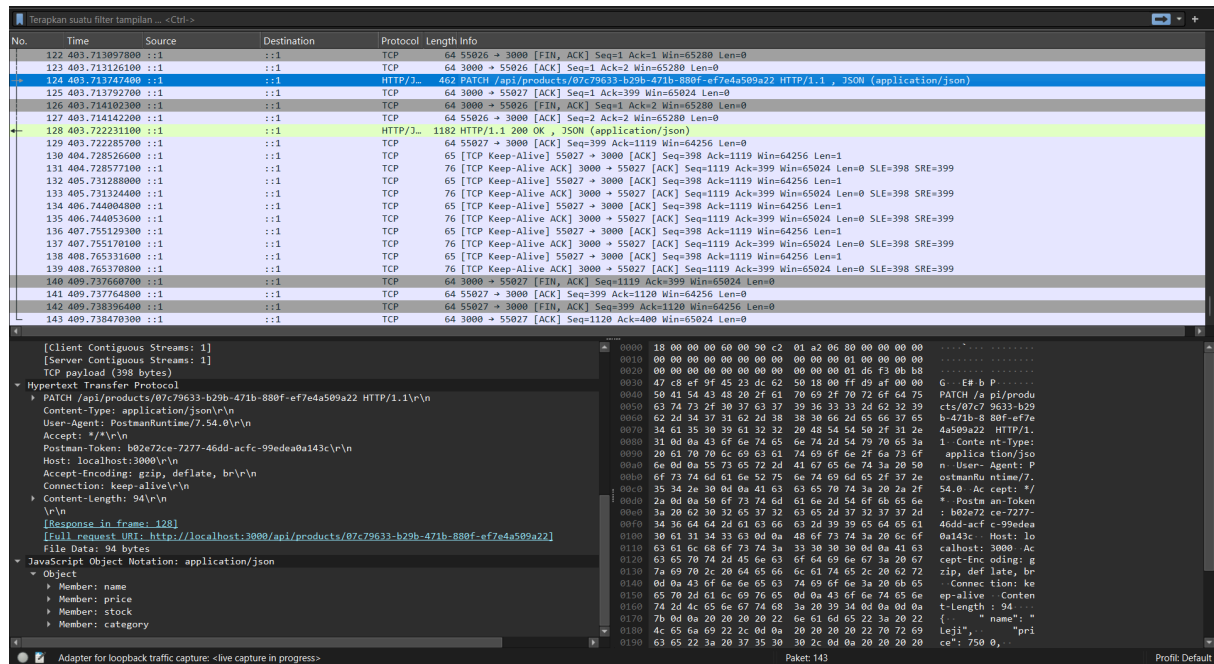
Adapter for loopback traffic capture: <live capture in progress> Packet: 115 Profile: Default

Terapkan suatu filter tampilan... <Ctrl>

No.	Time	Source	Destination	Protocol	Length	Info
122	403.713097800	:::1	:::1	TCP	64	55026 → 3000 [FIN, ACK] Seq=1 Ack=1 Win=65280 Len=0
123	403.713126100	:::1	:::1	TCP	64	3000 → 55026 [ACK] Seq=1 Ack=2 Win=65280 Len=0
124	403.713747400	:::1	:::1	HTTP/1.1	462	PATCH /api/products/07c79633-b29b-471b-880f-ef7e4509a22 HTTP/1.1, JSON (application/json)
125	403.714102300	:::1	:::1	TCP	64	3000 → 55027 [ACK] Seq=1 Ack=399 Win=65024 Len=0
126	403.714102300	:::1	:::1	TCP	64	3000 → 55026 [FIN, ACK] Seq=1 Ack=2 Win=65280 Len=0
127	403.714142200	:::1	:::1	TCP	64	55026 → 3000 [ACK] Seq=2 Ack=2 Win=65280 Len=0
128	403.722231100	:::1	:::1	HTTP/1.1	1182	HTTP/1.1 200 OK, JSON (application/json)
129	403.722357800	:::1	:::1	TCP	64	55027 → 3000 [ACK] Seq=399 Ack=1119 Win=64256 Len=0
130	404.728526600	:::1	:::1	TCP	65	[TCP Keep-Alive] 55027 → 3000 [ACK] Seq=398 Ack=1119 Win=64256 Len=1
131	404.728577100	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 55027 [ACK] Seq=1119 Ack=399 Win=65024 Len=0 SLE=398 SRE=399
132	405.731288000	:::1	:::1	TCP	65	[TCP Keep-Alive] 55027 → 3000 [ACK] Seq=398 Ack=1119 Win=64256 Len=1
133	405.731324400	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 55027 [ACK] Seq=1119 Ack=399 Win=65024 Len=0 SLE=398 SRE=399
134	406.744004800	:::1	:::1	TCP	65	[TCP Keep-Alive] 55027 → 3000 [ACK] Seq=398 Ack=1119 Win=64256 Len=1
135	406.744053600	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 55027 [ACK] Seq=1119 Ack=399 Win=65024 Len=0 SLE=398 SRE=399
136	407.755129300	:::1	:::1	TCP	65	[TCP Keep-Alive] 55027 → 3000 [ACK] Seq=398 Ack=1119 Win=64256 Len=1
137	407.755170100	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 55027 [ACK] Seq=1119 Ack=399 Win=65024 Len=0 SLE=398 SRE=399
138	408.765331600	:::1	:::1	TCP	65	[TCP Keep-Alive] 55027 → 3000 [ACK] Seq=398 Ack=1119 Win=64256 Len=1
139	408.765378000	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 55027 [ACK] Seq=1119 Ack=399 Win=65024 Len=0 SLE=398 SRE=399
140	409.737660700	:::1	:::1	TCP	64	3000 → 55027 [FIN, ACK] Seq=1119 Ack=399 Win=65024 Len=0
141	409.737764800	:::1	:::1	TCP	64	55027 → 3000 [ACK] Seq=399 Ack=1120 Win=64256 Len=0
142	409.738396400	:::1	:::1	TCP	64	55027 → 3000 [FIN, ACK] Seq=399 Ack=1120 Win=64256 Len=0
143	409.738470300	:::1	:::1	TCP	64	3000 → 55027 [ACK] Seq=1120 Ack=400 Win=65024 Len=0

[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]
TCP payload (398 bytes)
Hypertext Transfer Protocol
PATCH /api/products/07c79633-b29b-471b-880f-ef7e4509a22 HTTP/1.1\r\nContent-Type: application/json\r\nUser-Agent: PostmanRuntime/7.54.0\r\nAccept: */*\r\nPostman-Token: b02672ce-7277-46dd-acfc-99ede0a143c\r\nHost: localhost:3000\r\nAccept-Encoding: gzip, deflate, br\r\nConnection: keep-alive\r\nContent-Length: 94\r\n\r\n[Response in frame: 128]
[Full request URI: http://localhost:3000/api/products/07c79633-b29b-471b-880f-ef7e4509a22]
File Data: 94 bytes
JavaScript Object Notation: application/json
Object
Member: name
Member: price
Member: stock
Member: category

Adapter for loopback traffic capture: <live capture in progress> Packet: 143 Profile: Default



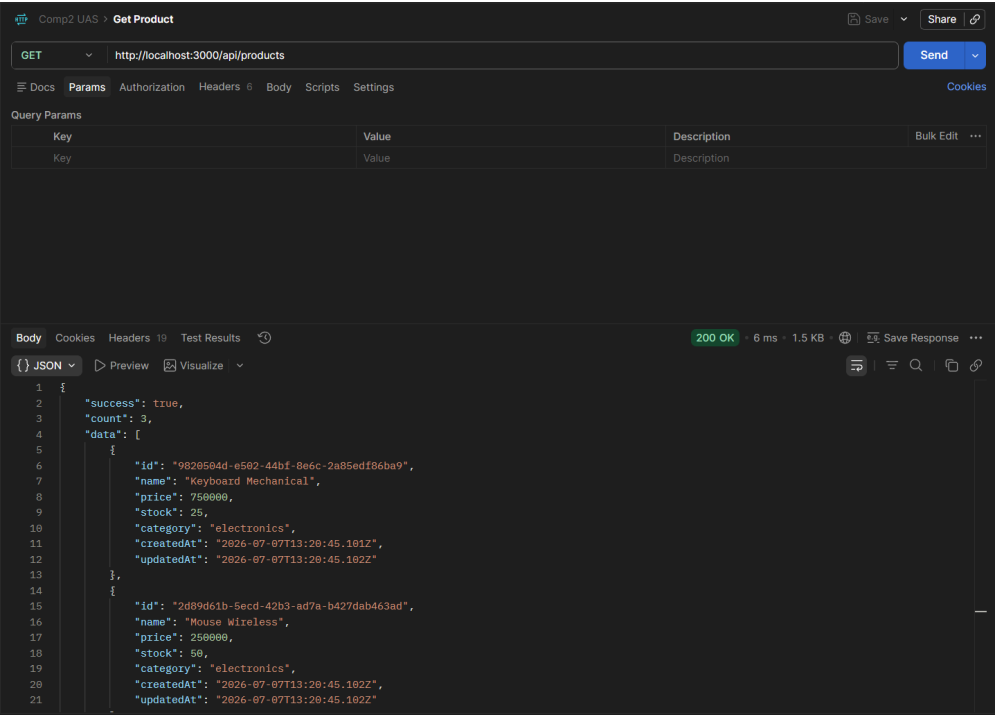
Testing result dan reliability analysis

Testing Result

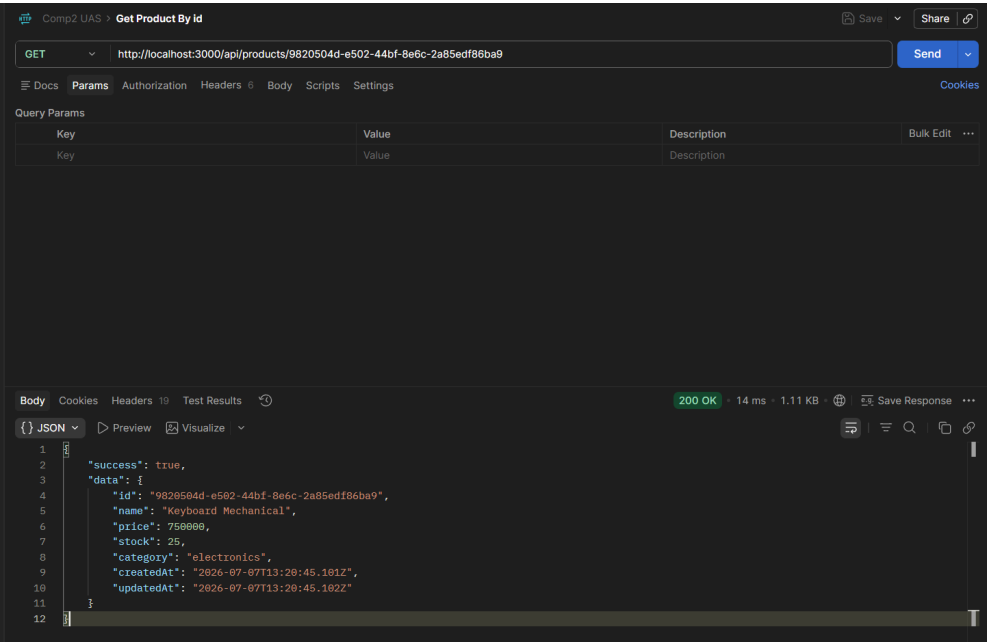
Pengujian dilakukan menggunakan aplikasi **Postman** untuk memastikan setiap endpoint REST API dapat berfungsi sesuai dengan spesifikasi yang telah dirancang. Pengujian meliputi operasi **Create (POST)**, **Read (GET)**, **Update (PUT)**, dan **Delete (DELETE)** pada data produk.

Hasil pengujian menunjukkan bahwa seluruh endpoint berhasil merespons request dengan status code yang sesuai. Endpoint **GET** berhasil menampilkan data produk, **POST** berhasil menambahkan data baru, **PUT** berhasil memperbarui data produk yang dipilih, dan **DELETE** berhasil menghapus data produk dari penyimpanan JSON. Selain pengujian skenario berhasil (*success case*), dilakukan juga pengujian terhadap skenario gagal (*error case*), seperti penggunaan ID yang tidak ditemukan atau data yang tidak valid. Pada kondisi tersebut, sistem berhasil mengembalikan pesan kesalahan beserta status code yang sesuai.

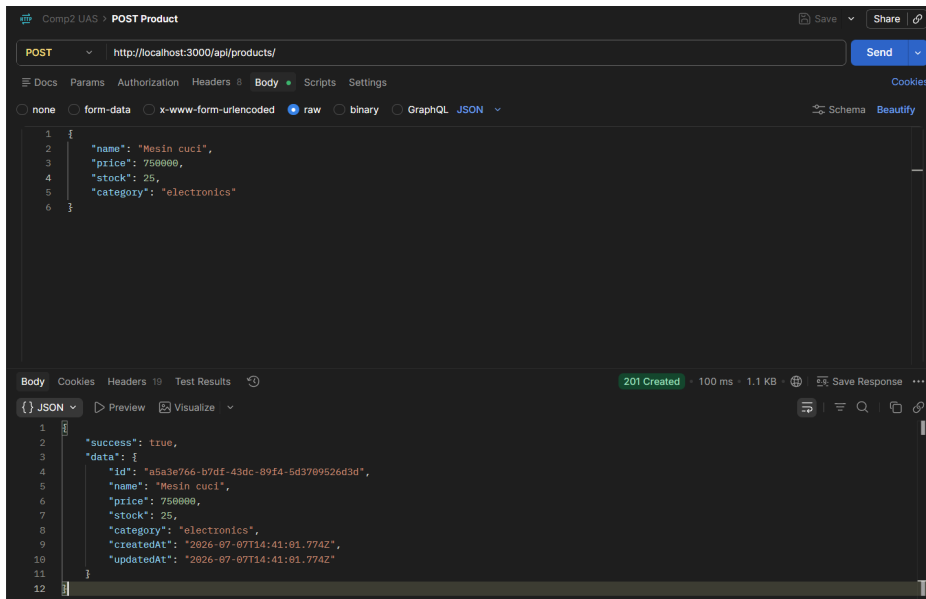
GET /api/products



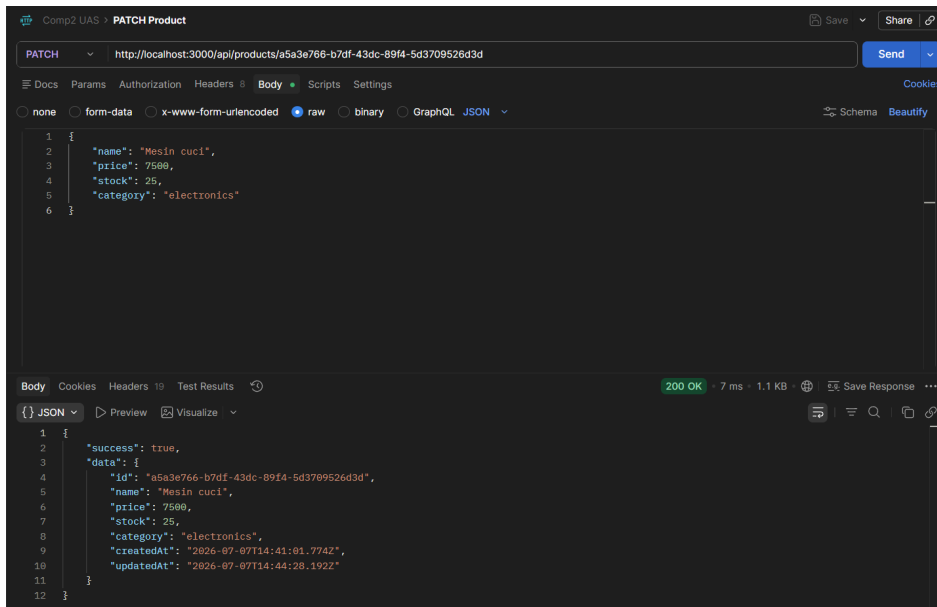
GET /api/products/id



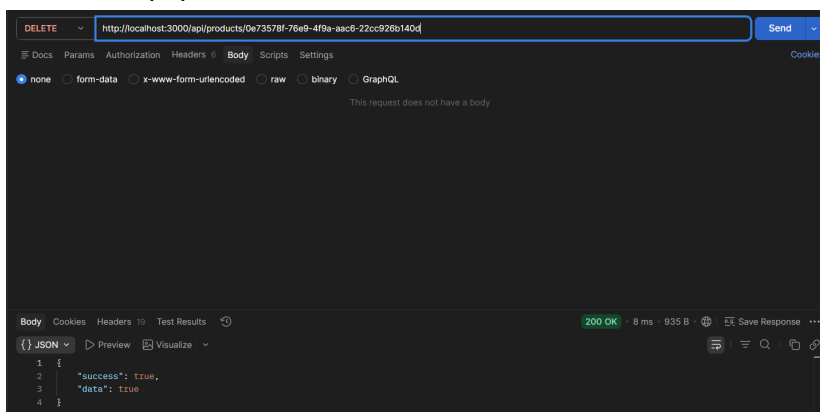
POST /api/products



PATCH /api/products/id



DELETE /api/products/id



Reliability Analysis

Berdasarkan hasil pengujian, REST API yang dikembangkan memiliki tingkat keandalan yang baik untuk kebutuhan pengelolaan data produk. Seluruh endpoint mampu memberikan respons yang sesuai dengan spesifikasi, baik pada kondisi normal maupun ketika terjadi kesalahan pada request.

Arsitektur aplikasi yang memisahkan **Router**, **Controller**, **Validation**, **Product Model**, dan **JSON Storage** juga meningkatkan kemudahan dalam proses pemeliharaan dan pengembangan sistem. Validasi data sebelum proses penyimpanan membantu mencegah data yang tidak valid masuk ke dalam sistem, sedangkan penggunaan HTTP status code yang sesuai memudahkan client dalam mengidentifikasi hasil dari setiap request.

Meskipun demikian, sistem masih memiliki keterbatasan karena menggunakan **JSON Storage** sebagai media penyimpanan data. Pendekatan ini cukup untuk kebutuhan mini project dan skala kecil, tetapi kurang optimal untuk aplikasi dengan jumlah data yang besar atau banyak pengguna secara bersamaan. Pada pengembangan selanjutnya, media penyimpanan dapat diganti menggunakan sistem basis data seperti MySQL atau MongoDB agar performa, keamanan, dan skalabilitas sistem menjadi lebih baik.

Kesimpulan, Limitation, Improvement, dan Reflection Pembelajaran

Kesimpulan

Mini project ini berhasil mengimplementasikan sebuah **REST API** untuk pengelolaan data produk menggunakan **Express.js**. API yang dikembangkan mendukung operasi **Create, Read, Update, dan Delete (CRUD)** melalui metode HTTP **GET, POST, PUT, dan DELETE**. Seluruh endpoint menghasilkan respons dalam format JSON dan telah diuji menggunakan Postman dengan hasil yang sesuai dengan spesifikasi yang telah dirancang.

Penerapan arsitektur yang memisahkan **Router**, **Controller**, **Validation**, **Product Model**, dan **JSON Storage** membuat struktur kode menjadi lebih terorganisir, mudah dipahami, serta mempermudah proses pengembangan dan pemeliharaan aplikasi.

Limitation

Meskipun API telah berjalan dengan baik, masih terdapat beberapa keterbatasan pada implementasi mini project ini. Penyimpanan data masih menggunakan **file JSON**, sehingga kurang optimal untuk menangani data dalam jumlah besar maupun akses secara bersamaan oleh banyak pengguna. Selain itu, sistem autentikasi dan otorisasi pengguna belum diterapkan sehingga seluruh endpoint masih dapat diakses tanpa mekanisme keamanan. Dokumentasi API juga masih terbatas dan belum menggunakan standar dokumentasi seperti OpenAPI atau Swagger.

Improvement

Pengembangan selanjutnya dapat dilakukan dengan mengganti media penyimpanan dari file JSON ke sistem basis data seperti **MySQL**, **PostgreSQL**, atau **MongoDB** agar pengelolaan data menjadi lebih efisien dan skalabel. Selain itu, API dapat ditingkatkan dengan menambahkan autentikasi menggunakan **JSON Web Token (JWT)**, validasi input yang lebih lengkap, dokumentasi API menggunakan Swagger, serta fitur pencarian, filter, dan pagination untuk meningkatkan pengalaman pengguna dan performa sistem.

Reflection Pembelajaran

Melalui mini project ini, penulis memperoleh pemahaman yang lebih baik mengenai konsep **REST API** dan penerapannya menggunakan **Express.js**. Proses pengembangan memberikan pengalaman dalam merancang endpoint, mengelola request dan response HTTP, melakukan validasi data, serta menerapkan struktur aplikasi yang modular. Selain itu, pengujian menggunakan Postman membantu memahami pentingnya memastikan setiap endpoint bekerja sesuai spesifikasi dan mampu menangani berbagai kondisi, baik pada skenario berhasil maupun gagal. Pengalaman ini menjadi dasar yang baik untuk mengembangkan aplikasi backend yang lebih kompleks pada proyek-proyek selanjutnya.